```
#import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


#Load the data
from google.colab import files
uploaded = files.upload()
df = pd.read_csv('Breast_Cancer_Databasee.csv')
df.head(7)
```

Choose Files Breast_Can…abasee.csv

- **Breast_Cancer_Databasee.csv**(application/vnd.ms-excel) - 127838 bytes, last modified: 7/14/2020 - 100% done

```
Saving Breast_Cancer_Databasee.csv to Breast_Cancer_Databasee (2).csv
```

	ID	Diagnosis	Radius mean	Texture_Mean	Perimeter_Mean	Area_Mean	Smoothness_Me
0	842302	M	17.99	10.38	122.80	1001.0	0.118
1	842517	M	20.57	17.77	132.90	1326.0	0.084
2	843009	M	19.69	21.25	130.00	1203.0	0.109
3	843483	M	11.42	20.38	77.58	386.1	0.142
4	843584	M	20.29	14.34	135.10	1297.0	0.100
5	843786	M	12.45	15.70	82.57	477.1	0.127
6	844359	M	18.25	19.98	119.60	1040.0	0.094

```
#Count the number of rows and columns in the data set
df.shape
```

```
(569, 33)
```

```
#Count the number of empty(NaN,NAN,na,0) values in each column
df.isna().sum()
```

```
ID                         0
Diagnosis                  0
Radius mean                0
Texture_Mean               0
Perimeter_Mean             0
Area_Mean                  0
Smoothness_Mean            0
Compactness_Mean           0
Concavity_Mean             0
Concave_Points_Mean        0
Symmetry_Mean              0
Fractal_Dimension_Mean     0
Radius_se                  0
Texture_se                 0
Perimeter_se               0
Area_se                    0
Smoothness_se              0
Compactness_se             0
Concavity_se               0
Concave points_se          0
Symmetry_se                0
```

```
#Drop the column with all missing values
df = df.dropna(axis=1)
```

```
Perimeter_worst            0
```

```
#Get the new count of the number of rows and columns
df.shape
```

```
(569, 32)
```

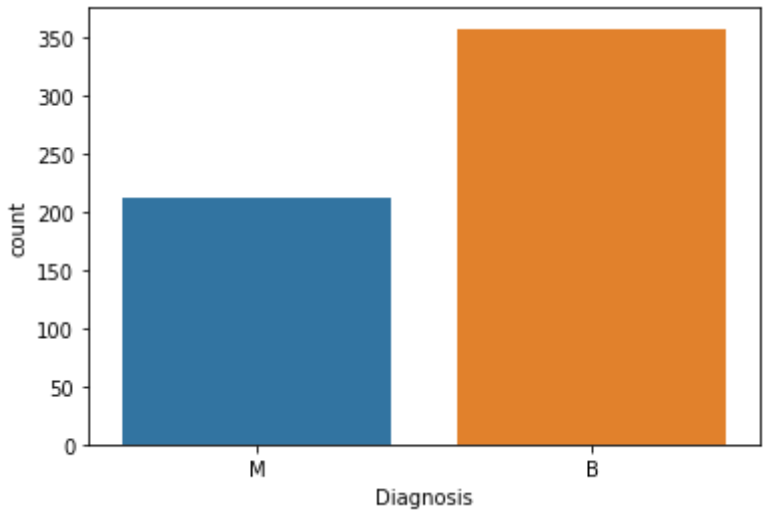
```
Symmetry_Worst             0
```

```
#Get a count of the number of Malignant(M) or Benign(B) cells
df['Diagnosis'].value_counts()
```

```
B    357
M    212
Name: Diagnosis, dtype: int64
```

```
#Visualize the count
sns.countplot(df['Diagnosis'],label='count')
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f1a158a2748>
```



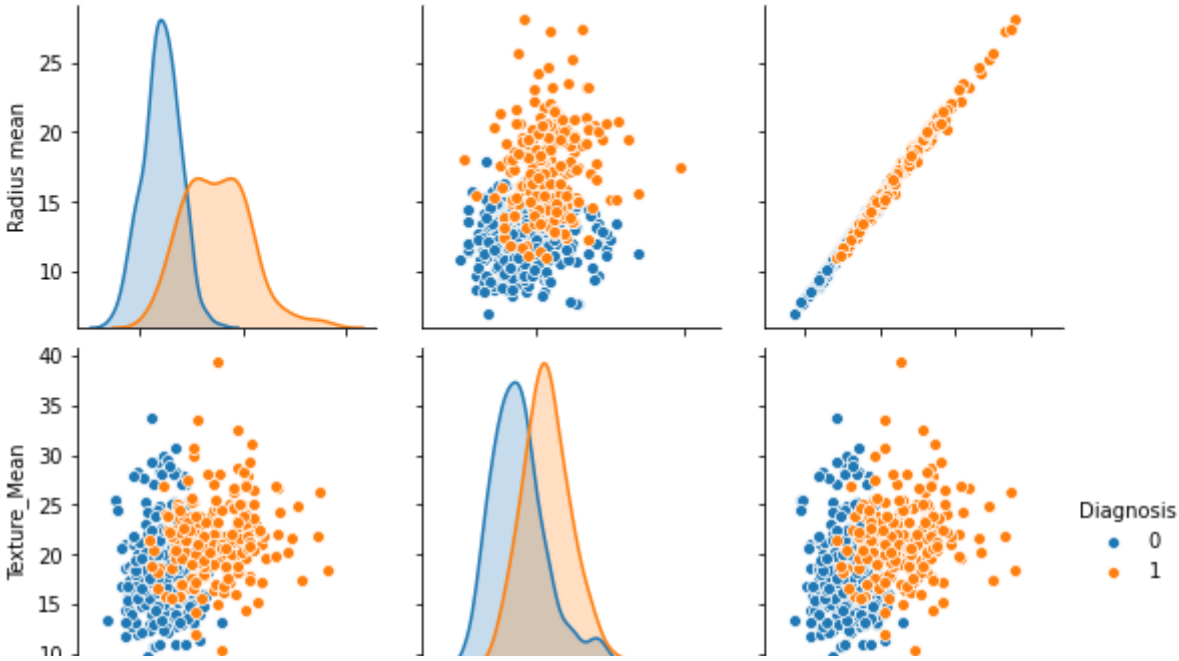
```
#Look at the data types to see which columns need to be encoded.
df.dtypes
```

```
ID                          int64
Diagnosis                  object
Radius mean               float64
Texture_Mean              float64
Perimeter_Mean            float64
Area_Mean                 float64
Smoothness_Mean           float64
Compactness_Mean          float64
Concavity_Mean            float64
Concave_Points_Mean       float64
Symmetry_Mean             float64
Fractal_Dimension_Mean    float64
Radius_se                 float64
Texture_se                float64
Perimeter_se              float64
Area_se                   float64
Smoothness_se             float64
Compactness_se            float64
Concavity_se              float64
Concave points_se         float64
Symmetry_se               float64
Fractal_Dimension_se      float64
Radius_worst              float64
Texture_worst             float64
Perimeter_worst           float64
Area_Worst                float64
Smoothness_Worst          float64
Compactness_Worst         float64
Concavity_Worst           float64
Concave_Points_Worst      float64
Symmetry_Worst            float64
Fractal_Dimension_Worst   float64
dtype: object
```

```
#Encode the categorical data values
from sklearn.preprocessing import LabelEncoder
labelencoder_Y = LabelEncoder()
df.iloc[:,1] = labelencoder_Y.fit_transform(df.iloc[:,1].values)
```

```
#Create a pair plot
sns.pairplot(df.iloc[:,1:5], hue='Diagnosis')
```

```
<seaborn.axisgrid.PairGrid at 0x7f1a0dd29e48>
```




```
#Print the 1st 5 rows of the new data
df.head(5)
```

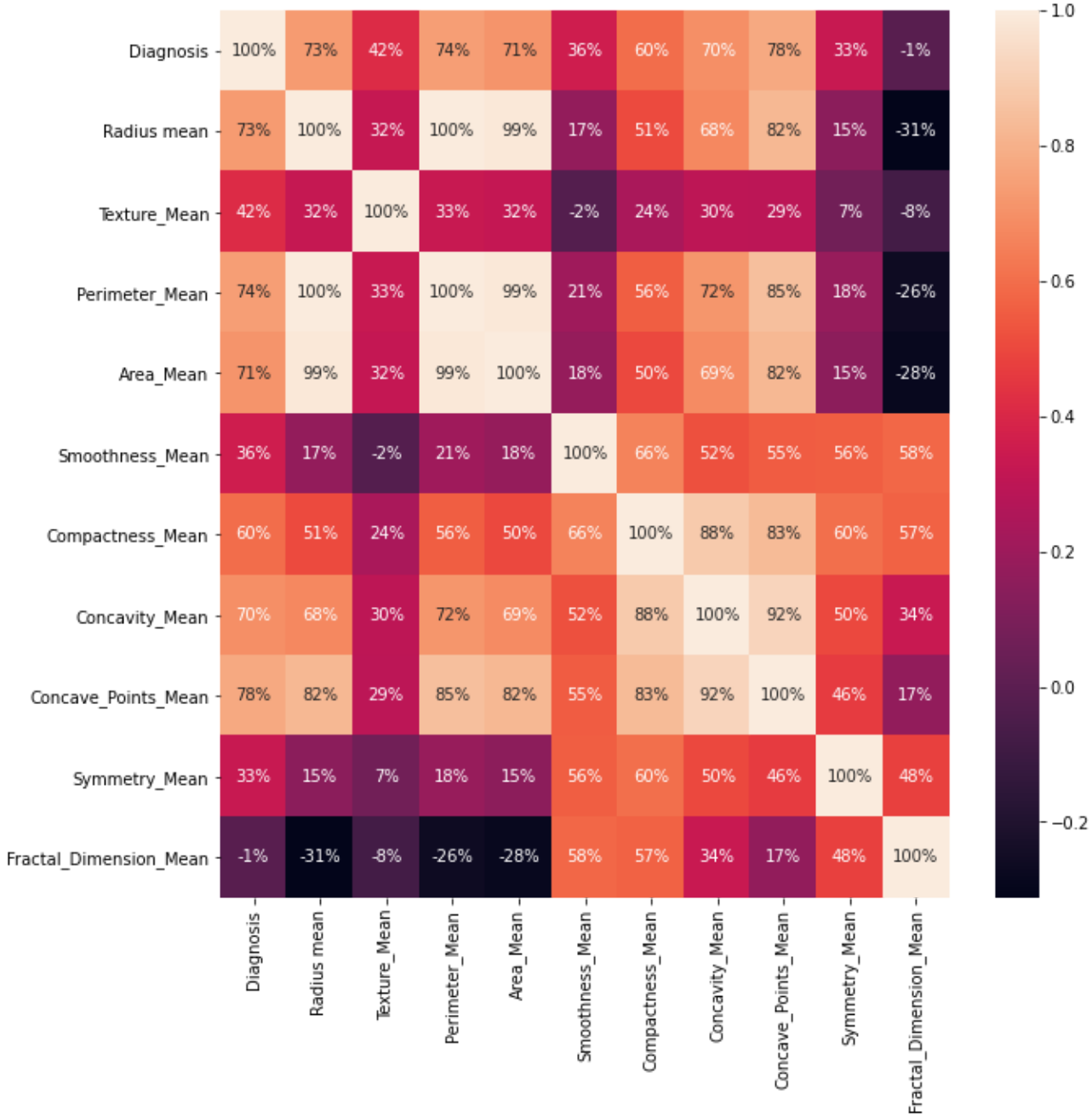
	ID	Diagnosis	Radius mean	Texture_Mean	Perimeter_Mean	Area_Mean	Smoothness_Mean
0	187	1	17.99	10.38	122.80	1001.0	0.11840
1	189	1	20.57	17.77	132.90	1326.0	0.08474
2	193	1	19.69	21.25	130.00	1203.0	0.10960
3	194	1	11.42	20.38	77.58	386.1	0.14250
4	195	1	20.29	14.34	135.10	1297.0	0.10030

```
#Get the correlation of the columns
df.iloc[:,1:12].corr()
```

	Diagnosis	Radius mean	Texture_Mean	Perimeter_Mean	Area_Mean
Diagnosis	1.000000	0.730029	0.415185	0.742636	0.708984
Radius mean	0.730029	1.000000	0.323782	0.997855	0.987357

```
#Visualize the correlation
plt.figure(figsize=(10,10))
sns.heatmap(df.iloc[:,1:12].corr(), annot=True, fmt='.0%')
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f1a0b9ec0f0>
```




```
#Split the data set into independent (x) and dependent (y) data sets
x = df.iloc[:,2:31].values
y = df.iloc[:,1].values
```

```
#Split the data set into 75% training and 25% testing
```

```
#Split the data set into 75% training and 25% testing
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.25 , random_state


#Scale the data (Feature scaling)
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.fit_transform(x_test)



#Create a function for the models
def models(x_train, y_train):

  #Logistic Regression
  from sklearn.linear_model import LogisticRegression
  log = LogisticRegression(random_state=0)
  log.fit(x_train, y_train)

  #Decision Tree
  from sklearn.tree import DecisionTreeClassifier
  tree = DecisionTreeClassifier(criterion = 'entropy', random_state=0)
  tree.fit(x_train, y_train)

  #Random Forest Classifier
  from sklearn.ensemble import RandomForestClassifier
  forest = RandomForestClassifier(n_estimators = 10, criterion = 'entropy', random_state =
  forest.fit(x_train, y_train)

  #Print the models accuracy on the training data
  print('[0]Logistic Regression Training Accuracy:', log.score(x_train, y_train))
  print('[1]Decision Tree Classifier Training Accuracy:', tree.score(x_train, y_train))
  print('[2]Random Forest Classifier Training Accuracy:', forest.score(x_train, y_train))

  return log, tree, forest



#Getting all of the models
model = models(x_train, y_train)
```

```
[0]Logistic Regression Training Accuracy: 0.9906103286384976
[1]Decision Tree Classifier Training Accuracy: 1.0
[2]Random Forest Classifier Training Accuracy: 0.9953051643192489
```

```
#Test model accuracy on data on confusion matrix
from sklearn.metrics import confusion_matrix

for i in range( len(model) ):
  print('Model ', i)
  cm = confusion_matrix(y_test, model[i].predict(x_test))

  TP = cm[0][0]
  TN = cm[1][1]
```

```
    FN = cm[1][0]
    FP = cm[0][1]

    print(cm)
    print('Testing Accuracy = ', (TP + TN)/(TP + TN + FN +FP))
    print()
```

```
Model  0
[[86  4]
 [ 3 50]]
Testing Accuracy =  0.951048951048951

Model  1
[[83  7]
 [ 2 51]]
Testing Accuracy =  0.9370629370629371

Model  2
[[87  3]
 [ 2 51]]
Testing Accuracy =  0.965034965034965
```

```
#Show another way to get metrics of the models
from sklearn.metrics import classification_report
from sklearn.metrics import accuracy_score

for i in range( len(model) ):
  print('Model ', i)
  print( classification_report(y_test, model[i].predict(x_test)))
  print( accuracy_score(y_test, model[i].predict(x_test)))
  print()
```

```
Model  0
              precision    recall  f1-score   support

           0       0.97      0.96      0.96        90
           1       0.93      0.94      0.93        53

    accuracy                           0.95       143
   macro avg       0.95      0.95      0.95       143
weighted avg       0.95      0.95      0.95       143

0.951048951048951
```

```
#Print the prediction of Random Forest Classifier Model
pred = model[2].predict(x_test)
print(pred)
print(y_test)
```

```
[1 0 0 0 0 0 0 0 0 0 1 0 0 1 1 1 0 1 1 1 1 1 0 0 1 0 0 1 0 1 0 1 0 1 0 1 0
 1 0 1 0 0 1 0 0 1 0 0 0 1 1 1 1 0 0 0 0 0 0 1 1 1 0 0 1 0 1 1 1 0 0 1 0 0
 1 0 0 0 0 0 1 1 1 0 1 0 0 0 1 1 0 1 0 1 0 0 1 0 0 0 0 0 0 0 1 0 1 0 1 1 0
 1 1 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 1 0 0 0 1]
[1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 1 1 1 0 0 1 0 0 1 0 1 0 1 0 1 0 1 0
 1 0 1 1 0 1 0 0 1 0 0 0 1 1 1 1 0 0 0 0 0 0 1 1 1 0 0 1 0 1 1 1 0 0 1 0 1
 1 0 0 0 0 0 1 1 1 0 1 0 0 0 1 1 0 1 0 1 0 0 1 0 0 0 0 0 0 0 1 0 1 0 1 1 0
 1 1 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 1 0 0 0 1]
              precision    recall  f1-score   support

           0       0.98      0.97      0.97        90
           1       0.94      0.96      0.95        53

    accuracy                           0.97       143
   macro avg       0.96      0.96      0.96       143
weighted avg       0.97      0.97      0.97       143

0.965034965034965
```